🛡️ Repo Shield Dashboard

Loaded report: report.md

Tip: click the ⋮ (three dots) menu in the top-right of the Streamlit app, choose 'Print', then use your browser's 'Save as PDF' option.

🛡️ Repo Shield Security Report

Repository: https://github.com/AtulYadavCodes/passwordgenerator
Scan Time: 2026-03-26
Total Findings: 15

📊 Executive Summary

This repository contains **15 potential security issues**, including:

- 🔴 0 Critical Issues (possible secrets / credentials)
- 🟠 2 High Risk Issues (unsafe patterns)
- 🟡 13 Medium Risk Issues (dependency risks)

👉 Immediate attention is required for **credential exposure and secret leakage**.

ℹ️ Note: These counts come from the static scanner/AST layer and represent *suspected* risk before AI review. See the 🤖 AI Audit section for confirmations and false-positive explanations.

📈 Risk Breakdown

Severity (pre-AI)	Count
🔴 Critical	0
🟠 High	2
🟡 Medium	13

🚨 Critical Issues

No critical issues found.

⚠️ High Risk Issues

Unsafe Pattern Usage

- **File:** package-lock.json
- **Reason:** High-entropy secret-like string detected (entropy=5.55)
- **File:** App.jsx
- **Reason:** High-entropy secret-like string detected (entropy=5.70)

📦 Dependency Risks (Aggregated)

⚠️ Found **13+ unpinned dependencies**

Example:

- @tailwindcss/vite@^4.1.11
- react@^19.1.0
- react-dom@^19.1.0

👉 Risk:

- Unexpected breaking changes
- Supply chain attacks

📋 Detailed Findings

#	File	Severity (pre-AI)	Reason
1	package-lock.json	🔴 High	High entropy string
2	package.json	🟡 Medium	Unpinned dependency
3	package.json	🟡 Medium	Unpinned dependency
4	package.json	🟡 Medium	Unpinned dependency
5	package.json	🟡 Medium	Unpinned dependency
6	package.json	🟡 Medium	Unpinned dependency
7	package.json	🟡 Medium	Unpinned dependency
8	package.json	🟡 Medium	Unpinned dependency
9	package.json	🟡 Medium	Unpinned dependency
10	package.json	🟡 Medium	Unpinned dependency
11	package.json	🟡 Medium	Unpinned dependency
12	package.json	🟡 Medium	Unpinned dependency
13	package.json	🟡 Medium	Unpinned dependency
14	package.json	🟡 Medium	Unpinned dependency
15	App.jsx	🔴 High	High entropy string

🤖 AI Audit

⚠️ Note: The following manifest/lock files are *not* sent to AI for auditing and are reported based solely on static scanner/AST analysis. Only manifest/lock files flagged as high risk (secrets / very high-entropy strings) are included in the AI audit. When these files show only medium or low risk issues (for example unpinned dependencies or integrity hashes in package-lock.json), they are generally considered safe metadata rather than true code-level vulnerabilities.

- package.json

- Files sent to AI audit: **2**
- Successful AI responses: **2**
- Failed AI audits: **0**

#	File	Status	AI Summary
1	package-lock.json	✅ Success	**1. Main Risk Summary**
2	App.jsx	✅ Success	#### 1. Main Risk Summary

🧠 Detailed AI Summaries (per file)

package-lock.json

Section	Details
Summary	### Security Audit Report **1. Main Risk Summary** The flagged high-entropy strings are **false positives**; they are standard cryptographic integrity hashes (Subresource Integrity) used by npm to verify package authenticity and do not represent leaked secrets or credentials. **2. Specific Issues** * **Issue:** High-entropy cryptographic hashes detected in `integrity` fields. These strings (SHA-512) naturally have high entropy but are public data intended for package verification. * ...

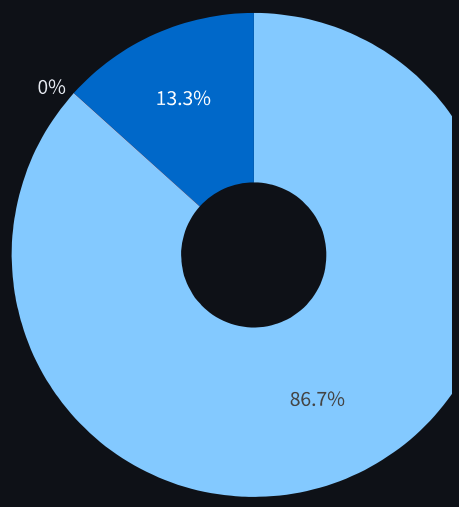
App.jsx

Section	Details
Main Risk	The application uses a cryptographically insecure random number generator (`Math.random()`) to generate passwords, making the output predictable and unsuitable for security purposes.
Specific Issues	* **Cryptographically Weak PRNG (Line 15):** The code uses `Math.floor(Math.random() * Str.length)` to select characters. `Math.random()` does not provide cryptographically strong symbols and is predictable by an attacker who can determine the internal state of the generator. * **Sensitive Data Exposure in UI (Line 36):** The generated password is displayed in an `<input type="text" />`. This allows "shoulder surfing" where the password is visible to anyone near the user's screen or via screen sharing. * **High-Entropy String (False Positive/Code Quality) (Line 13):** The string `"!@#$%^&*()_+[]{}|;:,.<>?"` triggered the high-entropy detector. While this is a character set and not a hardcoded secret (like an API key), hardcoding special character strings can sometimes lead to bypasses if the set is not comprehensive or contains characters that require escaping. * **Weak Default Security (Line 4):** The default password length is set to `4`, which is insufficient for any modern security requirement and susceptible to brute-force attacks.
Fix Suggestions	* **Use Web Crypto API:** Replace `Math.random()` with `window.crypto.getRandomValues()` to ensure cryptographic randomness. ```javascript // Replace the for loop logic with: const array = new Uint32Array(length); window.crypto.getRandomValues(array); for (let i = 0; i < length; i++) {

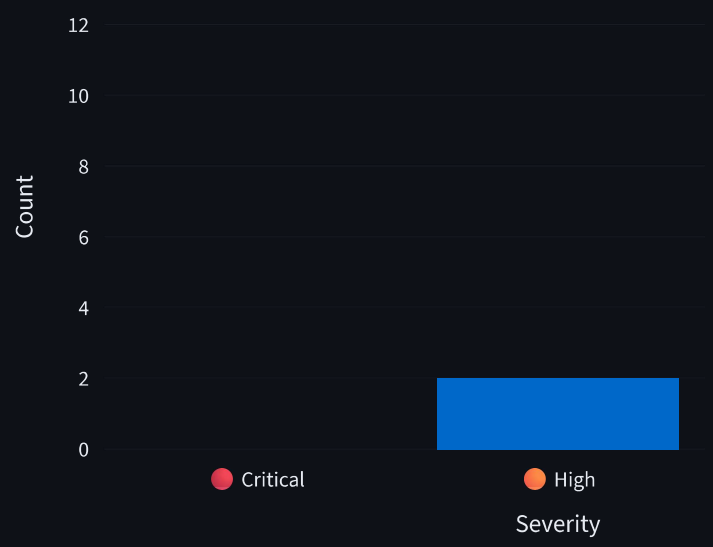
⚙️ Generated by Repo Shield

📊 Charts From Report

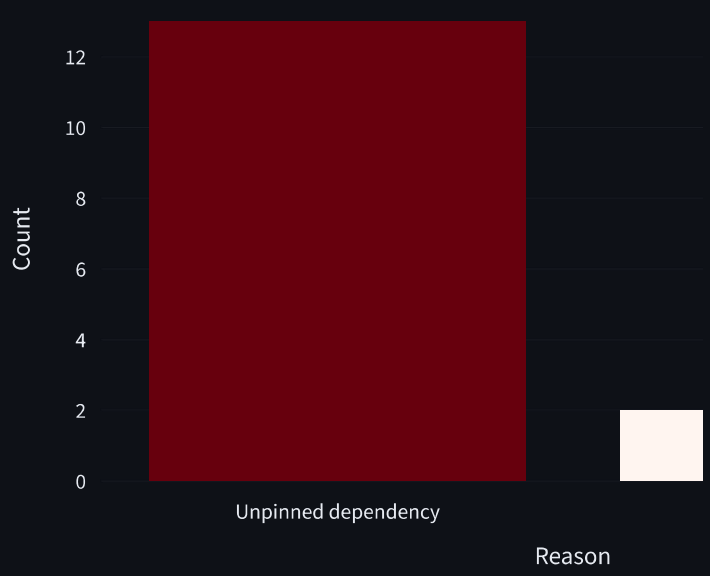
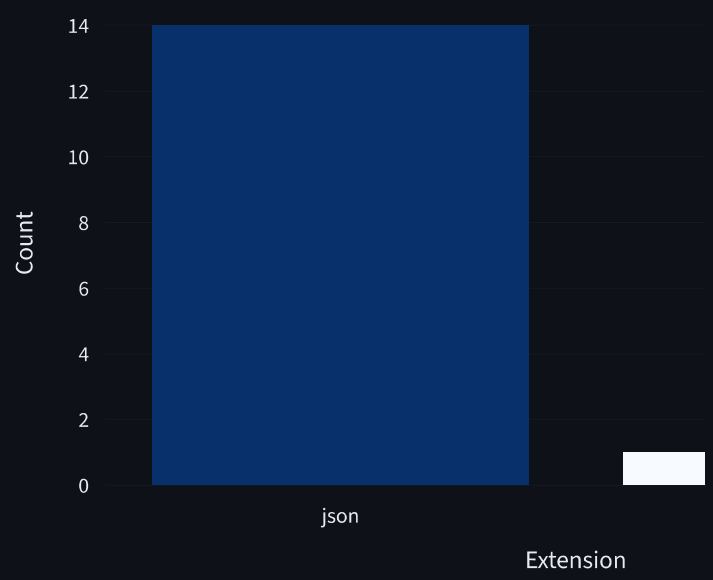
Risk Breakdown



Risk Counts by Severity



Top Finding Reasons



Findings by File Extension